Assignment 3
Hangman

by Jess Srinivas, Ben Grant, and Kerry Veenstra
CSE 13S, Fall 2026
Document version 1

Due Wednesday, October 28th, 2026, at 11:30 pm

Contents

1 Introduction **1**
2 The Game **1**
2.1 Initial Preparation — Copy the Resource Files 2
2.2 Rules 3
3 Background on Strings and Arrays **5**
3.1 Arrays 5
3.2 Strings 5
4 Your task **6**
4.1 Write is_lowercase_letter() 6
4.2 Write string_contains_character() 6
4.3 Write is_valid_secret() 6
4.4 Write prompt_for_and_read_character() 7
4.5 Write main() 7
4.6 Write run_hangman() 7
4.7 Testing, Testing, Testing 7
4.8 Submit: add/commit/push/submit commit ID 8
5 Revisions **8**

1 Introduction

Many of you have played the classic word game, Hangman (`https://en.wikipedia.org/wiki/Hangman_(game)`). Despite its morbid themes, hangman is sometimes played as a tool to teach vocabulary. In this assignment, you will be implementing the game in C.

2 The Game

Below we describe the features of the game. To help you see what the game should do, we have provided three executable files. (We often refers to these as "reference binaries".) Get the "reference binaries" by copying the starter files for the assignment.

2.1 Initial Preparation — Copy the Resource Files

☐ **To do:** Below are the steps for synchronizing the **resources** repository on your Github Codespaces instance and then copying the files for asgn3 into your repository. Remember that the you need to use the `cd` ("change directory") command to set the "current directory" of the Terminal. **The Explorer window does not affect the current directory of the Terminal.**

Note in the first `cd` command below, you want to go to `/workspaces/codespaces-blank` (notice that this "absolute path" starts with a slash `/`), but you can use "auto completion" to reduce the amount of typing that you need to do. That is, instead of typing

```
$ cd /workspaces/codespaces-blank ⏎
```

You can type the first few letters of a directory name followed by a tab key, as demonstrated below.

```
$ cd /w tab /c tab ⏎
$ cd 13s/resources
$ git pull
$ cd ..
$ cp resources/asgn3/* yourcruzid/asgn3
$ cd yourcruzid/asgn3
$ ls
```

You should see the three executable files ("reference binaries") called `hangman_arm`, `hangman_hub`, and `hangman_x86`. You can run one of these programs to see how a working hangman game should work.

If you are using Gibhub Codespaces, then run

```
$ ./hangman_hub "my secret phrase"        ⟵ Run this on Github Codespaces.
```

If you are running Ubuntu Multipass on a new Mac (with an M1, M2, M3, etc. processor), then run

```
$ ./hangman_arm "my secret phrase"        ⟵ Run this on Multipass on new Macs.
```

If you are running Multipass on an old Mac or Ubuntu under WSL and Windows, then run

```
$ ./hangman_x86 "my secret phrase"        ⟵ WSL on Windows and Multipass on old Macs.
```

If you try to run one of the reference binaries and see an error, then try to run another one.
(Later, you will be compiling your own program, and yours will be called `hangman`.)

☐ **To do:** Determine which reference binary will run on your laptop and play a game or two. If you want to end the game early, type control D or Ctrl D. (That is, while holding down control, type D. the control key is a kind of shift key in that you hold it down while pressing another key.)

☐ **To do:** While playing a game, make notes about what happens in these circumstances:

1. You guess a letter that is in the secret phrase.
2. You guess the same letter again.
3. You guess a letter that is *not* in the secret phrase.
4. You guess the same letter again.
5. You guess a digit.

6. You guess a capital letter.
7. You run the program with an uppercase secret phrase.
8. You run the program with digits in the secret phrase.
9. You run the program with a secret phrase that has spaces, hyphens (`-`), and apostrophes (`'`).
10. You try to guess spaces, hyphens, and apostrophes.

2.2 Rules

For this assignment, the hangman game that you create will have a few modifications:

1. The secret can be multiple words and can include apostrophes, spaces, and hyphens. An example secret phrase is `"don't go in empty-handed"`.
2. A secret *must* be provided and have at most 256 characters. (Meaning that the array that stores the string must have one more element—257 total—to fit a possible maximum-length string's terminating null byte.)
3. If special characters are present (space, `'`, and `-`), they will be shown at the start of the game (the player doesn't need to guess special characters—they are hints). For the secret phrase above, the program would print `"___'_ __ __ _____-______"`.
4. If only special characters are present, the player has already won.
5. The player will guess only one letter at a time followed by the Enter or return key. (We will use ⏎ to represent those keys.) For example, to guess the letters "a", "b", and "c", the user would type a ⏎ b ⏎ c ⏎.
6. You do not need to handle errors if the user enters more than one letter. For example, your program does **not** need to handle a b c ⏎.
7. If the user enters an invalid secret, you should print the exact error messages that are printed in the resources binary, and exit with any non-zero exit code.

We have provided you with a few strings in `hangman_strings.h`. For example, if you put `MSG_PROMPT` in your C code, it will be treated like `"Guess a letter: "`. That's because the file `hangman_strings.h` has a definition line like this:

```
#define MSG_PROMPT "Guess a letter: "
```

Use these like

```
    printf(MSG_PROMPT);
```

to write specific prompts to the screen. There is also an array of "ASCII art" called `arts[]` for hangman. Use it like

```
    printf("%s", arts[num_wrong_guesses]);
```

where the `int` variable `num_wrong_guesses` holds the number of wrong guesses that the user has made.

In the beginning of the game, you will print an empty gallows, a line for the phrase, and a list of eliminated letters. You will also print a new line and a prompt to have the player guess a letter.

```
  _____
 |     |
       |
       |
       |\
       | \
     __|__\

    Phrase: ___'_ __ __ _____-______
Eliminated:

Guess a letter:
```

In this example, the phrase is `don't go in empty-handed`. Note how the colons after "Phrase" and "Eliminated" align. Your output must match ours. The game will then prompt the player repeatedly to enter a character, and update the gallows, phrase, and eliminated sections. Any error handling for this section should match the binary provided to you. If the player loses, you will print the following

```
Guess a letter: b
  _____
 |     |
(_)    |
/|\    |
 |     |\
/ \    | \
     __|__\

    Phrase: d__'_ g_ i_ e____-h__ded
Eliminated: bcfjkl

You lose! The secret phrase was: don't go in empty-handed
```

In this example, the player typed the following keys:

l ⏎ k ⏎ j ⏎ i ⏎ h ⏎ g ⏎ f ⏎ e ⏎ d ⏎ c ⏎ b ⏎

(that is, they guessed letters in reverse alphabetical order, starting at lower-case L). However, note that the *eliminated letters print in alphabetical order*! So, independent of the order of the guesses, the list of eliminated letters always print in alphabetical order:

No: `Eliminated: lkjfbc`
Yes: `Eliminated: bcfjkl`

If a player guesses any letter that they have already guessed, **we should prompt the player for another letter without reprinting the gallows and phrase**. Here someone already has guessed `a` and `b`, and then they guess `a` and `b` again. The gallows is not reprinted.

```
  _____
 |     |
(_)    |
       |
       |\
       | \
     __|__\

    Phrase: ___'_ __ __ _____-_a____
Eliminated: b

Guess a letter: a
Guess a letter: b
Guess a letter:
```

© 2026 Kerry Veenstra

If the player wins, you will reprint the gallows and phrase, along with a message telling the player they won.

```
Guess a letter: a
  -----
 |     |
       |
       |
       |\
       | \
     __|__\

    Phrase: don't go in empty-handed
Eliminated:

You win! The secret phrase was: don't go in empty-handed
```

In this case, the player managed to guess the entire phrase without making any mistakes.

3 Background on Strings and Arrays

Hangman is a word game, and to create a word game, you must first have a representation for a word. You may recall that a `char` in C is actually just an 8-bit integer. To use that integer as a character, we use an ASCII table (which is only defined for positive numbers). The C programming language has a built-in ASCII table. That is, if you put a character in single-quotes, the C compiler interprets it as the int that is found in an ASCII table for that character. For example, in an ASCII table, the letter `a` is represented by the number 97. So if you put `'a '`in your C code, it will act like the int value 97.

A string is just an array of characters. To allow you to complete this assignment, we need to explain all of these in depth, and provide you tools to work with them.

3.1 Arrays

Arrays and memory are a topic that we will cover in a lot greater detail soon in this class, but we will cover the basics. Arrays in C work by storing data in sequential order in memory. When you write a line like, `int arr[3];` the space that the data is stored in is set aside when the program is compiled, meaning that you can't increase the size of an array.

Arrays can also implicitly convert to *pointers* (we'll go over these—they are like a memory address). In this case, you will end up with a pointer to the first element of the array. However, it's still important to keep in mind that arrays and pointers *are not the same thing*. One thing that demonstrates the difference is the `sizeof` operator, which tells you how much space a variable takes up in bytes. `sizeof` an array is the size of each element multiplied by the number of elements, whereas `sizeof` a pointer is a constant for each CPU architecture (8 on modern computers that use 64-bit processors).

3.2 Strings

Strings in C are just arrays of characters, followed by a null byte (a character whose decimal value is 0). The null byte is widely used by library functions to indicate the length of the string, so omitting a null terminator will likely lead to errors. Fortunately, the compiler automatically inserts null bytes for string literals. You can index into and iterate over a string just like an array. There is also a library for interacting with strings, `<string.h>`. For more information, see the `man` pages on this library, but as a hint, type the commands `man 3 strlen` and `man 3 strchr`.

© 2026 Kerry Veenstra

4 Your task

4.1 Write is_lowercase_letter()

☐ **To do:** Implement the function `is_lowercase_letter()`. What should the function do? Edit `hangman.c` and examine the starter function. The comment before the function in `hangman.c` describes what the function should do.

Important: in this assignment only you may not use `islower()` to implement this function. Its purpose is to demonstrate that you understand how to compare `char` values, such as `'a'` and `'z'`.

☐ **To do:** Compile your `hangman.c` program with this command:

```
$ clang -Wall -Werror hangman.c -o hangman
```

The two extra parameters direct the compiler to report common mistakes as errors:

- `-Wall` means "Report all common mistakes as warnings."
- `-Werror` means "Report warnings as errors."

☐ **To do:** Test the function `is_lowercase_letter()` by calling it in `main()` with a few test cases. Some example tests are below. *Note how calls to `is_lowercase_letter()` that are expected to return `0` or `false` invert the return code with `!`.*

```
assert(is_lowercase_letter('a'));
assert(!is_lowercase_letter('A'));
assert(!is_lowercase_letter('3'));
```

☐ **To do:** Once `is_lowercase_letter()` works, remove or comment out the tests.

4.2 Write string_contains_character()

☐ **To do:** Implement the function `string_contains_character()`. What should the function do? Edit `hangman.c` and examine the starter function. The comment before the function in `hangman.c` describes what the function should do.

☐ **To do:** Test the function `string_contains_character()` by calling it in `main()` with a few test cases. Some example tests are below. *Note how calls to `string_contains_character()` that are expected to return `0` or `false` invert the return code with `!`.*

```
assert(string_contains_character("abc", 'a'));
assert(string_contains_character("abc", 'b'));
assert(string_contains_character("abc", 'c'));
assert(!string_contains_character("abc", 'd'));
```

☐ **To do:** Once `string_contains_character()` works, remove or comment out the tests.

4.3 Write is_valid_secret()

☐ **To do:** Implement the function `is_valid_secret()`. What should the function do? Edit `hangman.c` and examine the starter function. The comment before the function in `hangman.c` describes what the function should do.

☐ **To do:** Test the function `is_valid_secret()` by calling it in `main()` with a few test cases. Some example tests are below. *Note how calls to `is_valid_secret()` that are expected to return `0` or `false` invert the return code with `!`.*

```
    assert(is_valid_secret("abcdefg-hijklmnop qrstuv'wxyz"));
    assert(!is_valid_secret("A"));
    assert(!is_valid_secret("3"));
```

☐ **To do:** Once `is_valid_secret()` works, remove or comment out the tests.

4.4 Write prompt_for_and_read_character()

☐ **To do:** Implement the function `prompt_for_and_read_character()`. (The file `hangman.c` has a starter function.) Here is the pseudocode for the function `prompt_for_and_read_character()`:

```
    prompt_for_and_read_character():
        print MSG_PROMPT  /* use printf() */

        do:
            get ch from user  /* use getchar() */
        while ch == '\n'

        if ch == EOF:
            exit(1)

        return ch
```

4.5 Write main()

☐ **To do:** Implement the function `main()`. What should the function do? Edit `hangman.c` and examine the starter function. The comment before the function in `hangman.c` describes what the function should do.

4.6 Write run_hangman()

☐ **To do:** Implement the function `run_hangman()`. What should the function do? In this case, review Section 2 of this PDF and execute the reference program.

4.7 Testing, Testing, Testing . . .

You should test your program using the test files that we provided (the first *To do* below), and you also should try to generate all of the error messages (the second *To do* below).

The `resources` repository has several files that you can use for testing. Use `tester.txt` as input for specific command lines (below). Then compare the output of your `./hangman` program with the resource files `expected_win.txt` and `expected_lose.txt`.

You already have seen the reference binaries `hangman_arm`, `hangman_hub`, and `hangman_x86`. You can use one of these to see exactly what output is expected from your own `hangman` program.

☐ **To do:** Test your program with the two tests below.

Here is a test command[1]. Notice that the command-line argument to `./hangman` is the letters of the alphabet in reverse order. Compare the result (`win.txt`) with `expected_win.txt`. (Remember `diff`?)

```
./hangman "zyxwvutsrqponmlkjihgfedcba" < tester.txt | head -1000 > win.txt
diff -u expected_win.txt win.txt
```

[1]The `head -1000` part of the command limits the size of output file to 1000 lines of text. We do this to avoid filling the storage disk when someone's program has an infinite loop.

© 2026 Kerry Veenstra

Here is another test command[1]. The command-line argument to `./hangman` is the phrase `"don't go in empty-handed"`. Compare the result (`lose.txt`) with `expected_lose.txt`.

```
./hangman "don't go in empty-handed" < tester.txt | head -1000 > lose.txt
diff -u expected_lose.txt lose.txt
```

☐ **To do:** Test your program more! For example, can you cause your program to generate all of its error messages? (We are going to check for this.)

4.8 Submit: add/commit/push/submit commit ID

You need to submit these files. You also should run the **git.ucsc.edu** pipeline.

- `hangman.c`
- `hangman_strings.h` (Yes, submit our copy of this file so that we can compile your `hangman.c`.)

☐ **To do:** `git add hangman.c hangman_strings.h`
☐ **To do:** `git commit -m 'whatever description of changes you want to mention'`
☐ **To do:** `git push`
☐ **To do:** Run the `asgn3` pipeline at **git.ucsc.edu**.
☐ **To do:** Submit your final commit ID to Canvas by Wednesday, October 28th at 11:30 pm.

5 Revisions

Version 1 Original.

© 2026 Kerry Veenstra